

DevOps Interview Master Skill v1.0

FAANG / Cisco / Meta / Zscaler / Palo Alto / Zoom / Oracle / Cloud / SaaS / Bay Area Startup Style Interview Preparation

Scope: beginner, intermediate, and advanced DevOps / SRE / Platform Engineering interview questions with strong answers, cheat sheets, workflows, diagrams, and memory tricks. Designed for Kubernetes, EKS, Argo CD, GitOps, GitHub Actions, runners, CloudBees/Jenkins, Helm deployments, CI/CD, Linux, networking, Terraform, observability, security, reliability, and production troubleshooting.

Note: This is not a leaked-question document. It is a maximum-coverage interview preparation bank based on the kinds of topics strong interviewers commonly probe across top technology companies and startups.

Level	Goal	Expected Interview Signal
Beginner	Explain core concepts and common commands clearly.	Can operate Linux, Git, basic CI/CD, Docker, Kubernetes basics.
Intermediate	Design, deploy, troubleshoot, and secure real systems.	Can own pipelines, Helm charts, EKS deployments, GitOps, incidents.
Advanced	Architect reliable platforms and reason about tradeoffs.	Can lead migrations, SRE programs, multi-account AWS, security, scaling, platform strategy.

How to use this PDF: First memorize the workflows and cheat sheets, then practice explaining every answer out loud in the STAR format: Situation, Task, Action, Result. For advanced roles, always add tradeoffs, failure modes, security implications, and metrics.

Table of Contents

1. Interviewer Mindset and Best Skills
2. Quick Architecture Diagrams
3. Beginner Question Bank
4. Intermediate Question Bank
5. Advanced Question Bank
6. Deep Coverage: Git, GitHub Actions, Runners, CloudBees/Jenkins
7. Deep Coverage: Docker, Helm, Kubernetes, EKS
8. Deep Coverage: Argo CD and GitOps
9. Deep Coverage: Terraform, AWS, IAM, Networking
10. Deep Coverage: Observability, SRE, Incident Response
11. Security, Secrets, Supply Chain, Compliance
12. Production Troubleshooting Workflows
13. Scenario and Design Rounds
14. Behavioral Questions and Strong Answers
15. Cheat Sheets and Memory Tricks

1. Interviewer Mindset and Best Skills

Top companies usually do not only test syntax. They test whether a candidate can reason under uncertainty, protect production, communicate clearly, and build systems that are secure, repeatable, observable, and scalable.

Skill Area	What strong interviewers look for	Strong answer pattern
Linux & Networking	Can debug CPU, memory, disk, DNS, TLS, ports, latency, packets.	Start broad, narrow with commands, validate with evidence.
CI/CD	Can build safe, fast, auditable pipelines with rollback.	Source -> build -> test -> scan -> package -> deploy -> verify.
Kubernetes/EKS	Understands workloads, networking, scheduling, autoscaling, security.	Explain control plane vs data plane, then app lifecycle.
GitOps/Argo CD	Can make Git the source of truth and handle drift.	Desired state in Git, controller reconciles, sync/health/prune/self-heal.
Cloud & IaC	Can build repeatable AWS infrastructure with Terraform.	Plan, review, state, modules, policy, apply, validate.
SRE	Can define SLOs, reduce toil, manage incidents, improve reliability.	SLI/SLO/error budget, alert quality, postmortem, automation.
Security	Least privilege, secrets, image scanning, SBOM, network policy.	Threat model, prevent, detect, respond, audit.
Leadership	Can influence teams, document, mentor, and drive standards.	Problem, constraints, tradeoffs, result, measurable impact.

Best skills to highlight on a resume or in interviews

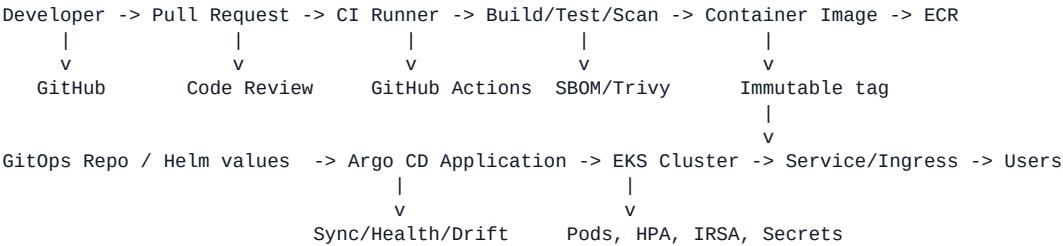
Kubernetes, Amazon EKS, Helm, Argo CD, GitOps, GitHub Actions, Jenkins/CloudBees, Terraform, AWS IAM/VPC/ALB/NLB/Route 53/ECR/EFS/CloudWatch, Docker, Linux, Bash/Python automation, Prometheus/Grafana, Splunk/ELK/OpenTelemetry, External Secrets, AWS Secrets Manager, IRSA, Karpenter/Cluster Autoscaler, RBAC, NetworkPolicy, TLS/cert-manager, incident response, SLO/SLI/error budgets, blue-green/canary deployments, secure SDLC, supply-chain security, platform engineering, self-service developer experience.

Trick: Organize skills as **Build -> Deploy -> Run -> Secure -> Scale**. This makes every interview answer sound structured.

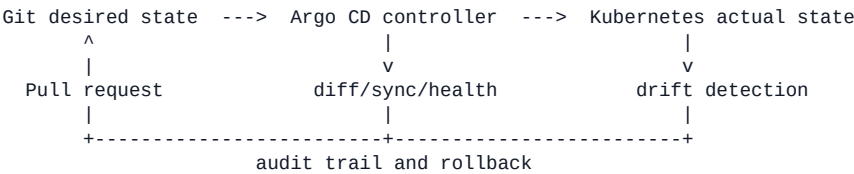
2. Quick Architecture Diagrams

Use these diagrams to quickly explain systems during interviews.

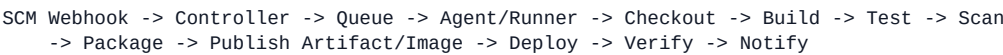
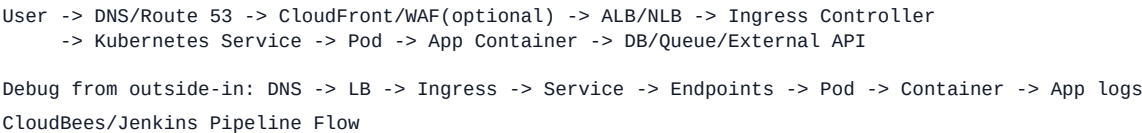
CI/CD to EKS Deployment Flow



GitOps Mental Model



EKS Request Path



Key risks: controller overload, stuck queue, bad credentials, workspace pollution, flaky tests, plugin vulnerabilities.

3. Beginner Question Bank

Q. What is DevOps?

Answer: DevOps is a culture and engineering practice that improves collaboration between development and operations so software can be delivered faster and more reliably. It combines automation, CI/CD, monitoring, infrastructure as code, feedback loops, and shared ownership.

Trick / clue: DevOps = people + process + automation + measurement.

Q. What is CI/CD?

Answer: CI means continuously integrating code changes with automated builds and tests. CD means continuously delivering or deploying validated changes to environments using repeatable pipelines.

Trick / clue: CI proves code works; CD safely moves it to users.

Q. What is the difference between continuous delivery and continuous deployment?

Answer: Continuous delivery keeps every change deployable but may require manual approval. Continuous deployment automatically releases changes to production after validations pass.

Trick / clue: Delivery = ready to deploy. Deployment = automatically deployed.

Q. What is Git?

Answer: Git is a distributed version control system that tracks code history, supports branching, merging, and collaboration.

Trick / clue: Git remembers every change and who made it.

Q. What is a pull request?

Answer: A pull request is a review workflow where code changes are proposed, discussed, tested, and merged after approval.

Trick / clue: PR = code + review + checks + audit trail.

Q. What is a merge conflict?

Answer: A merge conflict happens when Git cannot automatically combine changes because two branches changed the same lines or incompatible file sections.

Trick / clue: Conflict means Git needs a human decision.

Q. What is a GitHub Actions workflow?

Answer: A workflow is an automated pipeline defined in YAML under `.github/workflows`. It runs on triggers such as push, pull_request, schedule, or workflow_dispatch.

Trick / clue: Workflow = when + where + what steps.

Q. What is a runner?

Answer: A runner is the machine or container that executes CI/CD jobs. It can be GitHub-hosted or self-hosted.

Trick / clue: Runner = the worker that does the pipeline work.

Q. What is Docker?

Answer: Docker packages an application with its runtime dependencies into an image that can run consistently across environments as a container.

Trick / clue: Image is the recipe; container is the running meal.

Q. What is a Dockerfile?

Answer: A Dockerfile is a build recipe that defines base image, dependencies, files, commands, users, and entrypoint for creating a container image.

Trick / clue: Dockerfile = image blueprint.

Q. What is Kubernetes?

Answer: Kubernetes is a container orchestration platform that schedules containers, maintains desired state, handles service discovery, scaling, rolling updates, and self-healing.

Trick / clue: Kubernetes keeps containers running as declared.

Q. What is a Pod?

Answer: A Pod is the smallest deployable unit in Kubernetes. It contains one or more containers that share network and storage context.

Trick / clue: Pod = one app unit with shared IP.

Q. What is a Deployment?

Answer: A Deployment manages ReplicaSets and Pods, providing declarative rollout, rollback, and scaling behavior for stateless applications.

Trick / clue: Deployment owns replicas and rollout history.

Q. What is a Service in Kubernetes?

Answer: A Service provides a stable virtual IP and DNS name for accessing a changing set of Pods selected by labels.

Trick / clue: Service hides changing pod IPs.

Q. What is an Ingress?

Answer: Ingress defines HTTP/HTTPS routing from outside the cluster to internal Services, often implemented by an ingress controller such as NGINX, ALB controller, or APISIX.

Trick / clue: Ingress = HTTP traffic rules.

Q. What is Helm?

Answer: Helm is a Kubernetes package manager that uses charts to template and deploy related Kubernetes resources with configurable values.

Trick / clue: Helm = Kubernetes templates + package release.

Q. What is values.yaml?

Answer: values.yaml contains configuration inputs for a Helm chart. Templates use these values to render environment-specific manifests.

Trick / clue: values.yaml = knobs for the chart.

Q. What is Argo CD?

Answer: Argo CD is a GitOps continuous delivery tool for Kubernetes. It compares desired state in Git with actual cluster state and reconciles drift.

Trick / clue: Argo CD = Git-to-cluster sync engine.

Q. What is GitOps?

Answer: GitOps is an operating model where Git stores the desired state of infrastructure and applications, and controllers automatically reconcile that state to environments.

Trick / clue: Git is the source of truth.

Q. What is Terraform?

Answer: Terraform is an infrastructure-as-code tool that declares cloud resources, calculates plans, stores state, and applies changes predictably.

Trick / clue: Terraform = desired cloud state + state file.

Q. What is a Terraform state file?

Answer: State maps Terraform configuration to real infrastructure resources. It lets Terraform know what exists and what changed.

Trick / clue: State is Terraform memory.

Q. What is an AWS VPC?

Answer: A VPC is an isolated virtual network in AWS where subnets, route tables, security groups, NAT gateways, and internet gateways define connectivity.

Trick / clue: VPC = your private AWS network boundary.

Q. What is IAM?

Answer: IAM manages identities, permissions, roles, and policies for accessing AWS resources.

Trick / clue: IAM answers who can do what on which resource.

Q. What is an EKS cluster?

Answer: EKS is AWS managed Kubernetes. AWS manages the control plane while users manage node groups, networking, add-ons, workloads, and access policies.

Trick / clue: EKS = Kubernetes control plane as managed AWS service.

Q. What is a Kubernetes namespace?

Answer: A namespace is a logical partition in a cluster used to separate resources, teams, environments, and RBAC boundaries.

Trick / clue: Namespace = folder-like isolation, not hard security by itself.

Q. What is RBAC?

Answer: Role-Based Access Control defines what users or service accounts can do through Roles, ClusterRoles, RoleBindings, and ClusterRoleBindings.

Trick / clue: RBAC = verbs + resources + subjects.

Q. What is a ConfigMap?

Answer: A ConfigMap stores non-sensitive configuration data for Pods as environment variables, files, or command-line arguments.

Trick / clue: ConfigMap = non-secret config.

Q. What is a Secret?

Answer: A Secret stores sensitive data such as passwords or tokens, but should still be integrated with secure secret managers and encryption.

Trick / clue: Secret is sensitive config, not magic security.

Q. What is a liveness probe?

Answer: A liveness probe checks whether a container is stuck and should be restarted.

Trick / clue: Liveness = should I restart it?

Q. What is a readiness probe?

Answer: A readiness probe checks whether a Pod is ready to receive traffic.

Trick / clue: Readiness = should traffic be sent here?

Q. What is a startup probe?

Answer: A startup probe gives slow-starting applications time before liveness checks begin.

Trick / clue: Startup = do not kill me while booting.

Q. What is horizontal pod autoscaling?

Answer: HPA adjusts Pod replica count based on metrics such as CPU, memory, or custom metrics.

Trick / clue: HPA scales pods, not nodes.

Q. What is the difference between HPA and Cluster Autoscaler/Karpenter?

Answer: HPA changes application replicas. Cluster Autoscaler or Karpenter changes worker node capacity when Pods cannot be scheduled.

Trick / clue: HPA = pods; Karpenter = nodes.

Q. What is observability?

Answer: Observability is the ability to understand system behavior using telemetry such as metrics, logs, traces, events, and profiles.

Trick / clue: Metrics say what; logs say why; traces say where.

Q. What is a metric?

Answer: A metric is a numeric time-series measurement such as CPU usage, latency, request count, or error rate.

Trick / clue: Metric = number over time.

Q. What is a log?

Answer: A log is a timestamped event record from an application or system.

Trick / clue: Log = story of events.

Q. What is a trace?

Answer: A trace follows a request across services, showing spans, latency, and service dependencies.

Trick / clue: Trace = request journey.

Q. What is an SLO?

Answer: A Service Level Objective is a measurable reliability target, such as 99.9% successful requests under 300 ms.

Trick / clue: SLO = reliability promise you measure.

Q. What is an error budget?

Answer: An error budget is the allowed unreliability within an SLO. It balances feature velocity and reliability work.

Trick / clue: Budget spent = slow down releases and fix reliability.

Q. What is a rollback?

Answer: Rollback means reverting to a previously known good version when a deployment causes problems.

Trick / clue: Rollback is the emergency reverse gear.

4. Intermediate Question Bank

Q. How would you design a CI/CD pipeline for a microservice deployed to EKS?

Answer: Use PR checks for lint, unit tests, dependency scan, IaC validation, Docker build, image scan, SBOM, and integration tests. Publish immutable image tags to ECR. Update Helm values or GitOps manifests through a PR. Argo CD syncs to dev, then promotion to stage/prod with approvals, smoke tests, health checks, metrics validation, and rollback plan.

Trick / clue: Say: source -> test -> build -> scan -> publish -> GitOps -> deploy -> verify -> rollback.

Q. How do you deploy an application with Helm?

Answer: Create or reuse a chart, define templates for Deployment, Service, Ingress, ConfigMap, Secret references, HPA, PDB, and ServiceAccount. Put environment-specific settings in values files. Run helm lint, helm template, helm upgrade --install, then verify rollout and app health.

Trick / clue: Helm flow = lint, template, install/upgrade, status, rollback.

Q. What is the difference between helm install, helm upgrade, and helm rollback?

Answer: helm install creates a release. helm upgrade changes an existing release, often with --install for idempotency. helm rollback returns a release to a previous revision stored in Helm history.

Trick / clue: Install new, upgrade change, rollback undo.

Q. How do you troubleshoot a failed Helm deployment?

Answer: Run helm status, helm history, helm get manifest/values, kubectl describe events, kubectl get pods, logs, rollout status, and inspect rendered templates using helm template. Check image pull, probes, secrets, RBAC, resource limits, and namespace mismatches.

Trick / clue: Debug in layers: Helm -> Kubernetes objects -> Pod -> app.

Q. How does Argo CD detect drift?

Answer: Argo CD continuously compares live Kubernetes objects against rendered desired manifests from Git/Helm/Kustomize. Differences show as OutOfSync. It can alert, manually sync, or auto-sync with self-heal and prune.

Trick / clue: Desired vs live diff = drift.

Q. What does Argo CD sync policy mean?

Answer: Sync policy controls how Argo CD applies desired state. Manual sync requires human action. Automated sync applies changes automatically. Prune removes resources no longer in Git. Self-heal reverts manual drift.

Trick / clue: Auto, prune, self-heal are the three dangerous/powerful switches.

Q. What is the difference between Argo CD Application and ApplicationSet?

Answer: Application deploys one app definition. ApplicationSet generates multiple Applications from templates using generators such as list, git, cluster, or matrix, useful for multi-cluster/multi-environment deployments.

Trick / clue: ApplicationSet = factory for Applications.

Q. How do GitHub Actions runners differ from CloudBees/Jenkins agents?

Answer: Both execute jobs. GitHub runners are tied to GitHub workflow jobs and can be hosted or self-hosted. Jenkins/CloudBees agents connect to controllers and execute pipeline stages. CloudBees adds enterprise governance, shared libraries, RBAC, and managed controllers.

Trick / clue: Runner/agent = worker; controller/orchestrator schedules work.

Q. How do you secure self-hosted runners?

Answer: Use ephemeral runners where possible, isolate networks, restrict repo access, avoid running untrusted fork PRs with secrets, rotate tokens, patch images, run least privilege IAM, clean workspaces, disable privileged Docker unless needed, and monitor runner activity.

Trick / clue: Treat runners as production servers with secrets exposure risk.

Q. What is a GitHub Actions matrix strategy?

Answer: Matrix runs a job across combinations of variables such as OS, language version, architecture, or region. It increases coverage while keeping workflow YAML concise.

Trick / clue: Matrix = one job template, many combinations.

Q. How do you pass artifacts between CI jobs?

Answer: Upload build artifacts from one job and download them in dependent jobs. For containers, publish immutable images to a registry and pass the image tag or digest.

Trick / clue: Artifacts for files; registries for images.

Q. How do you prevent a broken pipeline from deploying to production?

Answer: Use branch protection, required checks, approval gates, environment protection rules, canary/staged rollout, image scanning, policy checks, and health verification before promotion.

Trick / clue: Protect prod with checks + approvals + progressive delivery.

Q. What is image immutability and why does it matter?

Answer: Images should be promoted by digest or immutable tag so the exact tested artifact is what reaches production. Mutable tags like latest create audit and rollback risk.

Trick / clue: Build once, promote the same artifact.

Q. What is Kubernetes rolling update?

Answer: A rolling update gradually replaces old Pods with new Pods according to maxSurge and maxUnavailable, preserving availability during deployment.

Trick / clue: Rolling = swap pods gradually.

Q. How would you do blue-green deployment?

Answer: Run blue and green versions separately. Send traffic to current blue. Validate green with tests and shadow traffic, then switch routing. Keep blue available for quick rollback.

Trick / clue: Blue-green = two full environments, traffic switch.

Q. How would you do canary deployment?

Answer: Deploy a small percentage of traffic to the new version, monitor errors/latency/business metrics, then gradually increase traffic or roll back.

Trick / clue: Canary = small blast radius first.

Q. What is a PodDisruptionBudget?

Answer: PDB limits voluntary disruptions by requiring a minimum available or maximum unavailable number of Pods during maintenance or node drains.

Trick / clue: PDB protects availability during planned disruption.

Q. What are Kubernetes requests and limits?

Answer: Requests reserve CPU/memory for scheduling. Limits cap maximum usage. Bad values can cause pending Pods, throttling, or OOMKills.

Trick / clue: Requests schedule; limits enforce.

Q. Why can CPU limits hurt latency?

Answer: CPU limits can cause CFS throttling even when node CPU is available, increasing tail latency. Many production systems set CPU requests but avoid strict CPU limits for latency-sensitive workloads.

Trick / clue: CPU limit can throttle healthy apps.

Q. What causes OOMKilled?

Answer: A container exceeds its memory limit and the kernel kills it. Check limits, memory leaks, heap settings, traffic spikes, and sidecar usage.

Trick / clue: OOM = memory limit exceeded or node pressure.

Q. How do you troubleshoot CrashLoopBackOff?

Answer: Check pod logs --previous, describe pod events, command/args, env vars, config, secrets, probes, image version, dependency connectivity, and resource limits.

Trick / clue: CrashLoop = app starts then dies; logs --previous is key.

Q. How do you troubleshoot ImagePullBackOff?

Answer: Check image name/tag, registry credentials, ECR permissions, image existence, architecture compatibility, network access, pull secrets, and rate limits.

Trick / clue: ImagePull = registry/auth/name/network problem.

Q. How do you troubleshoot Pending Pods?

Answer: Check scheduler events for insufficient CPU/memory, taints/tolerations, node selectors, affinity, PVC binding, quotas, and autoscaler/Karpenter status.

Trick / clue: Pending = scheduler cannot place it.

Q. What is IRSA in EKS?

Answer: IAM Roles for Service Accounts maps a Kubernetes service account to an AWS IAM role using OIDC, allowing Pods to access AWS APIs without node-wide credentials.

Trick / clue: IRSA = pod-level AWS identity.

Q. How do External Secrets work?

Answer: External Secrets Operator reads secret values from external providers like AWS Secrets Manager and creates Kubernetes Secrets for Pods to consume.

Trick / clue: ExternalSecret = sync external secret into Kubernetes.

Q. How do you rotate secrets safely?

Answer: Rotate at source, sync to Kubernetes, restart or reload applications that do not watch secret changes, validate connectivity, and keep rollback/dual credential window if possible.

Trick / clue: Secret rotation = update source + reload consumers.

Q. What is cert-manager?

Answer: cert-manager automates TLS certificate issuance, renewal, and management in Kubernetes using issuers such as ACME, private CA, or Vault.

Trick / clue: cert-manager = certificate lifecycle controller.

Q. What is an AWS ALB Ingress Controller / AWS Load Balancer Controller?

Answer: It watches Kubernetes Ingress or Service resources and provisions AWS ALBs/NLBs with target groups, listeners, rules, and security group configuration.

Trick / clue: Controller turns Kubernetes intent into AWS load balancers.

Q. What is the difference between ALB and NLB?

Answer: ALB is layer 7 HTTP/HTTPS with host/path routing and WAF integration. NLB is layer 4 TCP/UDP/TLS with high performance and static IP support.

Trick / clue: ALB = HTTP smart; NLB = TCP fast.

Q. How does DNS troubleshooting work?

Answer: Check domain, resolver, authoritative records, TTL, CNAME/A records, split-horizon DNS, CoreDNS health, and use dig/nslookup from client and inside cluster.

Trick / clue: DNS debug: ask who resolves what and where.

Q. What is CoreDNS in Kubernetes?

Answer: CoreDNS provides DNS service discovery inside the cluster, resolving Services and Pods based on Kubernetes records.

Trick / clue: CoreDNS = cluster phonebook.

Q. What is a Kubernetes NetworkPolicy?

Answer: NetworkPolicy controls allowed ingress and egress traffic for Pods. It requires a CNI plugin that enforces policies.

Trick / clue: Default is open unless policy and CNI enforce it.

Q. What is Terraform plan?

Answer: terraform plan compares configuration, state, and real provider data to show intended changes before applying.

Trick / clue: Plan is the preview; apply is the change.

Q. How do you manage Terraform state safely?

Answer: Use remote backend like S3 with locking via DynamoDB or Terraform Cloud, restrict access, version state, avoid manual edits, and use workspaces/accounts carefully.

Trick / clue: State needs locking, encryption, versioning, access control.

Q. What is the difference between Terraform module and workspace?

Answer: A module is reusable configuration. A workspace is a state separation mechanism for the same configuration, often less preferred than explicit environment directories for complex production.

Trick / clue: Module = reusable code; workspace = state namespace.

Q. How do you avoid Terraform drift?

Answer: Use Git-based changes, regular plans, avoid console/manual edits, import unmanaged resources, policy enforcement, and alert on drift.

Trick / clue: Drift happens when reality changes outside IaC.

Q. What is least privilege?

Answer: Grant only permissions needed for a specific task, limited by resource, action, condition, and duration.

Trick / clue: Least privilege = no extra power.

Q. How do you design alerts?

Answer: Alert on symptoms users feel, not every cause. Use SLO burn rate, severity, runbooks, ownership, deduplication, and actionable thresholds.

Trick / clue: Good alert = urgent, actionable, owned.

Q. What are golden signals?

Answer: Latency, traffic, errors, and saturation are key service health signals.

Trick / clue: L-T-E-S: Latency, Traffic, Errors, Saturation.

Q. What is a postmortem?

Answer: A blameless document analyzing incident timeline, impact, root causes, contributing factors, detection gaps, and corrective actions.

Trick / clue: Postmortem asks how the system failed, not who failed.

Q. How do you reduce toil?

Answer: Identify repetitive manual work, automate safely, improve self-service, standardize templates, add guardrails, and measure time saved.

Trick / clue: Toil is manual, repetitive, automatable, tactical work.

5. Advanced Question Bank

Q. Design a production-grade EKS platform for multiple teams.

Answer: Use separate AWS accounts/environments, VPC/subnet design, private endpoints where possible, EKS managed control plane, node groups or Karpenter, IRSA, External Secrets, ECR, ALB/NLB ingress, cert-manager, observability stack, policy engine, network policies, backup/DR, GitOps with Argo CD, environment promotion, platform templates, RBAC, quotas, and self-service onboarding. Include cost, security, reliability, and upgrade strategy.

Trick / clue: Answer with layers: account -> network -> cluster -> platform add-ons -> app delivery -> operations.

Q. How would you migrate from OpenShift to EKS?

Answer: Inventory workloads, routes, security context constraints, storage, secrets, CI/CD, operators, and observability. Build EKS landing zone and baseline add-ons. Map OpenShift Routes to Ingress/ALB, SCC to Pod Security/RBAC, internal registry to ECR, secrets to AWS Secrets Manager/External Secrets. Run parity tests, dual deploy, traffic migration, rollback, and decommission.

Trick / clue: Migration = discover, map, build, parallel run, cutover, validate, retire.

Q. Explain GitOps failure modes and mitigations.

Answer: Failure modes include bad manifests, accidental prune, secret drift, CRD ordering, controller outage, manual hotfix drift, repo outage, wrong cluster context, and auto-sync blast radius. Mitigate with PR review, policy checks, sync waves, progressive rollout, app-of-apps safeguards, environment protections, limited prune, backups, and clear break-glass process.

Trick / clue: GitOps is safe only with guardrails.

Q. How do you design Argo CD for many clusters?

Answer: Use hub-and-spoke Argo CD or per-cluster Argo CD depending on blast radius. Use ApplicationSets with cluster generators, RBAC per team, SSO, projects, repo allowlists, sync windows, resource exclusions, monitoring, high availability, and secret management for cluster credentials.

Trick / clue: Multi-cluster Argo = scale plus blast-radius control.

Q. How would you secure Argo CD?

Answer: Enable SSO/RBAC, least-privilege projects, restrict source repos/destinations, avoid admin tokens, rotate cluster credentials, require signed commits if possible, limit plugins, protect repo branches, audit syncs, restrict exec/log access, and secure Redis/API server ingress with TLS.

Trick / clue: Argo CD can deploy anything, so protect it like prod admin.

Q. Explain Helm chart design best practices for enterprise applications.

Answer: Use helpers for naming/labels, schema validation, default values, environment overlays, securityContext, resources, probes, HPA, PDB, serviceAccount, annotations, ingress, secrets references, config checksums for rollout, and avoid hardcoding cluster-specific values.

Trick / clue: A good chart is configurable but not chaotic.

Q. How do you handle CRDs in Helm and Argo CD?

Answer: CRDs must exist before custom resources. Use crds/ directory or separate CRD app, sync waves, and careful upgrade process because Helm does not upgrade CRDs like normal templates. In Argo CD, split CRDs and CRs or use sync waves/hooks.

Trick / clue: CRDs first, CRs second.

Q. Design a secure GitHub Actions platform for an enterprise.

Answer: Use org policies, branch protections, required workflows, OIDC federation to cloud, least-privilege environments, ephemeral self-hosted runners, runner groups, artifact retention, dependency review, secret scanning, pinned actions, reusable workflows, approval gates, and audit logs.

Trick / clue: Enterprise CI = guardrails + reuse + identity federation.

Q. Why is OIDC better than long-lived cloud secrets in GitHub Actions?

Answer: OIDC lets workflows request short-lived cloud credentials based on repository, branch, environment, and workflow claims. This avoids storing static AWS keys in GitHub secrets and reduces blast radius.

Trick / clue: OIDC = temporary identity, not static keys.

Q. How do you design self-hosted runners on Kubernetes?

Answer: Use an operator or scale set, ephemeral pods per job, isolated namespaces, resource quotas, network policies, minimal runner images, no privileged mode unless required, cache strategy, log collection, autoscaling, and cleanup guarantees.

Trick / clue: Every job should get a clean room.

Q. How would you troubleshoot a slow CI pipeline?

Answer: Break down queue time, checkout time, dependency install, build, tests, artifact upload, scans, and deployment. Add caching, parallelism, matrix tuning, incremental builds, test splitting, faster runners, and remove serial bottlenecks.

Trick / clue: Measure first. Optimize the longest stage.

Q. CloudBees/Jenkins controller is overloaded. How do you fix it?

Answer: Inspect queue, executors, heap, garbage collection, plugin health, job concurrency, logs, and controller CPU/memory. Move builds to agents, reduce polling, use webhooks, archive less, clean workspaces, split controllers, update plugins, and tune JVM.

Trick / clue: Controller orchestrates; agents do work.

Q. How do you implement progressive delivery on Kubernetes?

Answer: Use Argo Rollouts, Flagger, or service mesh. Define canary steps, traffic weights, analysis templates, Prometheus metrics, abort conditions, and rollback. Combine with feature flags and SLO-based checks.

Trick / clue: Progressive delivery = deploy + measure + decide.

Q. How do you design rollback for database migrations?

Answer: Prefer backward-compatible expand/contract migrations. Deploy code that supports old and new schema, add columns/tables first, backfill, switch reads/writes, then remove old schema later. Avoid destructive changes in same release.

Trick / clue: DB rollback is hard; design forward-compatible changes.

Q. How do you perform zero-downtime deployments?

Answer: Use readiness probes, rolling updates, graceful shutdown, connection draining, PDB, enough replicas, backward-compatible APIs, database expand/contract, and traffic shifting.

Trick / clue: Zero downtime requires app, infra, and data compatibility.

Q. How would you debug intermittent 5xx errors in EKS?

Answer: Check ingress/load balancer logs, target group health, service endpoints, pod restarts, readiness failures, app logs, latency/error metrics by route, resource throttling, network policies, DNS, dependencies, and recent deployments. Correlate with timestamps.

Trick / clue: Follow request path and correlate time.

Q. How would you debug high latency after deployment?

Answer: Compare before/after metrics, look at p95/p99, CPU throttling, GC, memory, DB calls, downstream latency, connection pools, DNS, retries, sidecars, HPA behavior, and rollout version differences.

Trick / clue: Latency = app + dependency + resource + network.

Q. How do you design observability for a microservices platform?

Answer: Standardize metrics, logs, traces, exemplars, dashboards, service catalog, SLOs, alert routing, synthetic checks, runbooks, correlation IDs, OpenTelemetry, and cost/retention policies.

Trick / clue: Observe the user journey, not just containers.

Q. Explain RED and USE methods.

Answer: RED for services: Rate, Errors, Duration. USE for resources: Utilization, Saturation, Errors. Together they cover user-facing health and infrastructure bottlenecks.

Trick / clue: RED services, USE resources.

Q. How do you define SLOs for an API?

Answer: Choose user-critical journeys, define SLIs like successful requests and latency under threshold, set targets such as 99.9%, calculate error budget, alert on burn rate, and review periodically.

Trick / clue: SLO starts with user pain.

Q. How do you handle noisy alerts?

Answer: Remove non-actionable alerts, tune thresholds, use aggregation, route by ownership, add inhibition/silencing, shift to SLO burn alerts, and create runbooks.

Trick / clue: Every page must deserve waking someone up.

Q. How do you design a secrets platform for Kubernetes?

Answer: Use AWS Secrets Manager or Vault as source, External Secrets Operator for sync, IRSA for access, KMS encryption, rotation process, namespace RBAC, audit logs, secret reload/restart strategy, and avoid secrets in Git.

Trick / clue: Secrets need source, sync, access, rotation, and reload.

Q. What is the difference between sealed secrets, external secrets, and CSI secrets store?

Answer: Sealed Secrets encrypts secrets into Git-safe objects decrypted by cluster controller. External Secrets syncs from external secret managers into Kubernetes Secrets. CSI Secrets Store mounts external secrets as volumes and may sync to K8s Secret optionally.

Trick / clue: Sealed = encrypted in Git; External = sync from manager; CSI = mount at runtime.

Q. How do you handle Kubernetes upgrades safely?

Answer: Review release notes and deprecated APIs, upgrade non-prod first, update add-ons, drain nodes gradually, validate workloads, monitor metrics, keep backups, and plan rollback/recreate strategy.

Trick / clue: Upgrade = APIs, add-ons, nodes, workloads.

Q. How do you prepare for a Kubernetes API deprecation?

Answer: Scan manifests with tools, check kubectl api-resources, update Helm charts and CRDs, test in staging, and block deprecated APIs in CI.

Trick / clue: Deprecations fail at upgrade time if ignored.

Q. How would you design network segmentation in EKS?

Answer: Use VPC/subnet segmentation, security groups for nodes and pods where applicable, NetworkPolicies, namespace isolation, private endpoints, egress controls, WAF, ingress allowlists, and service-to-service auth.

Trick / clue: Segment at cloud, cluster, and app layers.

Q. How do you reduce EKS cost?

Answer: Right-size requests, use Karpenter consolidation, Spot for tolerant workloads, HPA/VPA, scale-to-zero for non-prod, reduce log/metric cardinality, choose instance families, review load balancers/PVs, and enforce quotas.

Trick / clue: Cost = compute + storage + data + observability.

Q. Explain Karpenter vs Cluster Autoscaler.

Answer: Cluster Autoscaler scales predefined node groups. Karpenter provisions right-sized nodes directly based on pending pods and constraints, often faster and more flexible with consolidation.

Trick / clue: CA grows groups; Karpenter creates nodes for pods.

Q. How do you design multi-account AWS for platform engineering?

Answer: Use separate accounts for prod/non-prod/shared/network/security/logging, AWS Organizations, SCPs, centralized IAM Identity Center, shared VPC or transit gateway patterns, centralized logging, and account vending.

Trick / clue: Accounts are blast-radius boundaries.

Q. How do you secure Terraform in CI/CD?

Answer: Use remote state locking/encryption, plan in PR, policy as code, restricted apply permissions, separate plan/apply roles, OIDC credentials, secret scanning, module pinning, and manual approval for prod.

Trick / clue: Terraform can destroy prod; gate it heavily.

Q. What is policy as code?

Answer: Policy as code expresses governance rules in executable form using tools like OPA/Gatekeeper, Kyverno, Sentinel, or Conftest, preventing insecure or non-compliant changes.

Trick / clue: Policy becomes automated guardrails.

Q. How do you evaluate platform maturity?

Answer: Measure deployment frequency, lead time, change failure rate, MTTR, SLO attainment, developer onboarding time, toil hours, cost efficiency, security findings, and self-service adoption.

Trick / clue: Maturity = outcomes, not tools.

Q. How would you build a developer self-service platform?

Answer: Provide golden paths, templates, Helm charts, Terraform modules, service catalog, automated CI/CD, GitOps onboarding, observability defaults, RBAC, docs, and support channels with guardrails.

Trick / clue: Self-service = freedom with paved roads.

Q. How do you handle production break-glass access?

Answer: Use temporary, audited, approved elevated access with MFA, time-bound roles, session logging, post-use review, and automation to revoke access.

Trick / clue: Break glass should be rare, logged, and temporary.

Q. How would you secure software supply chain?

Answer: Use protected branches, code review, signed commits/tags, pinned dependencies/actions, SBOM, SAST/DAST, container scanning, provenance/SLSA, image signing, admission policies, and trusted registries.

Trick / clue: Supply chain = source, build, dependency, artifact, deploy.

Q. What is admission control in Kubernetes?

Answer: Admission controllers validate or mutate API requests before objects are persisted. Examples include Pod Security Admission, Kyverno, Gatekeeper, image policy webhooks, and mutating sidecar injection.

Trick / clue: Admission = gate before resource enters cluster.

Q. How do you troubleshoot webhook failures blocking deployments?

Answer: Check webhook service endpoints, TLS certs, DNS, namespace, failurePolicy, controller pod health, network policies, and API server events. Temporarily adjust failurePolicy only with risk approval.

Trick / clue: Broken webhooks can block the whole cluster.

Q. How do you design disaster recovery for EKS workloads?

Answer: Back up manifests/Git, persistent volumes, databases, secrets metadata, and cluster add-on configuration. Define RTO/RPO, multi-AZ, cross-region replication for data, recreate cluster via IaC, and run DR drills.

Trick / clue: Kubernetes state is rebuildable; data is precious.

Q. How do you explain a production incident to executives?

Answer: State customer impact, timeline, current status, mitigation, ETA, risk, and next update time. Avoid deep technical noise unless asked.

Trick / clue: Exec update = impact, action, ETA, confidence.

5A. Maximum Coverage Topicwise Question Bank

This section adds rapid-fire company-style open-ended questions. Practice answering each in 60-120 seconds. For senior interviews, add tradeoffs, failure modes, and metrics.

Linux and Shell

Q. How do you check disk usage on a Linux server?

Answer: Use `df -h` for filesystem capacity and `du -sh` or `du -h --max-depth=1` for directory-level usage. If disk is full, check logs, container images, core dumps, temp files, and deleted files held by processes with `ls -l +L1`.

Trick / clue: `df` = filesystem; `du` = directory; `ls -l +L1` = deleted-but-open files.

Q. How do you investigate high load average?

Answer: Check `uptime/top`, CPU usage, runnable processes, `iowait`, blocked tasks, disk latency, and process-level utilization. High load with low CPU often indicates I/O wait or blocked processes.

Trick / clue: Load is waiting work, not always CPU.

Q. How do you find which process is using a port?

Answer: Use `ss -ltnp` or `ls -ltnp`. Then map PID to service logs, systemd unit, container, or Kubernetes pod as needed.

Trick / clue: Port -> PID -> process -> owner.

Q. How do you troubleshoot permission denied errors?

Answer: Check file owner/group/mode, directory execute bit, SELinux/AppArmor, container user, mounted volume permissions, Kubernetes securityContext, and ACLs.

Trick / clue: Permission needs every parent directory to allow traversal.

Q. What is the difference between hard link and soft link?

Answer: A hard link points to the same inode. A soft link points to a path. Hard links cannot usually cross filesystems and do not work for directories; soft links can break if target path disappears.

Trick / clue: Hard = inode; soft = path.

Q. How do you monitor logs in Linux?

Answer: Use `journalctl` for systemd services, `tail -f` for files, `grep/awk` for filtering, and centralized log tools for production. For containers, use `docker logs` or `kubectl logs`.

Trick / clue: Local logs first, centralized logs for history/correlation.

Q. What is systemd?

Answer: systemd is a Linux init and service manager. It starts services, tracks status, restarts failed units, and manages logs through journald.

Trick / clue: systemd = service supervisor.

Q. How do you write a safe shell script?

Answer: Use `set -euo pipefail` carefully, quote variables, validate inputs, use temporary files safely, log actions, handle errors, and make destructive operations explicit.

Trick / clue: Quote variables and fail safely.

Q. How do you debug cron job failures?

Answer: Check cron schedule/user/environment, PATH, permissions, working directory, logs/mail, script exit code, and whether dependencies require interactive shell variables.

Trick / clue: Cron has a tiny environment.

Q. How do you check memory usage?

Answer: Use `free -h`, `top/htop`, `ps aux --sort=-%mem`, `smem` if available, and check cgroup/container limits. Understand cache/buffers are reclaimable.

Trick / clue: Linux uses free RAM for cache; not all used memory is bad.

Q. What is inode exhaustion?

Answer: Filesystem can run out of inodes even when bytes remain. `df -i` shows inode usage. Usually caused by too many small files.

Trick / clue: Disk has space and inode limits.

Q. How do you troubleshoot slow disk I/O?

Answer: Use iostat, await/util metrics, dmesg, cloud volume metrics, filesystem saturation, and noisy neighbor checks. Mitigate by scaling IOPS, moving logs, or reducing sync writes.

Trick / clue: Disk bottleneck shows as high iowait/await.

Networking, DNS, TLS

Q. Explain TCP three-way handshake.

Answer: Client sends SYN, server replies SYN-ACK, client replies ACK. After that the connection is established.

Trick / clue: SYN, SYN-ACK, ACK.

Q. How do you debug connection refused?

Answer: Connection refused means the host is reachable but nothing is listening on the target port or firewall actively rejected it. Check service process, listener, port, bind address, and security rules.

Trick / clue: Refused = reached host, no listener.

Q. How do you debug connection timeout?

Answer: Timeout usually means packets are dropped or no route. Check DNS, route, firewall/security group/NACL, network policy, load balancer, and target health.

Trick / clue: Timeout = packet disappeared.

Q. What is the difference between 4xx and 5xx?

Answer: 4xx usually means client/request problem such as auth, forbidden, not found, or validation. 5xx means server-side failure or upstream dependency failure.

Trick / clue: 4xx client side, 5xx server side.

Q. What is TLS termination?

Answer: TLS termination decrypts HTTPS at a load balancer, ingress, proxy, or application. After termination, traffic may be re-encrypted or sent internally depending on security requirements.

Trick / clue: Terminate where cert/private key lives.

Q. How do you check a certificate from CLI?

Answer: Use openssl s_client -connect host:443 -servername host and inspect certificate chain, expiration, SAN, issuer, and protocol/cipher.

Trick / clue: openssl sees what the client sees.

Q. What is DNS TTL?

Answer: TTL controls how long resolvers cache a DNS record. Lower TTL helps migrations but increases DNS query volume.

Trick / clue: TTL = cache lifetime.

Q. What is split-horizon DNS?

Answer: The same DNS name resolves differently depending on where the query comes from, such as internal vs external network.

Trick / clue: Same name, different answers.

Q. What is MTU and why does it matter?

Answer: MTU is maximum packet size. MTU mismatch can cause fragmentation or black-hole connectivity, especially with VPNs, overlays, and jumbo frames.

Trick / clue: MTU bugs look like weird partial connectivity.

Q. Explain Layer 4 vs Layer 7 load balancing.

Answer: Layer 4 balances TCP/UDP connections without understanding HTTP. Layer 7 understands HTTP details like host/path/header and can do smarter routing.

Trick / clue: L4 = connection; L7 = application.

Q. What is health check in load balancer?

Answer: Health checks determine whether targets should receive traffic. Bad health check paths, ports, security rules, or readiness can remove healthy pods or keep bad ones active.

Trick / clue: Health check controls traffic eligibility.

Q. How do retries cause outages?

Answer: Retries can amplify load during partial failures, causing retry storms. Use timeouts, backoff, jitter, circuit breakers, and retry budgets.

Trick / clue: Retries are load multipliers.

Docker and Containers

Q. What is the difference between image and container?

Answer: An image is an immutable filesystem/package template. A container is a running instance of that image with process, network, and writable layer.

Trick / clue: Image is class; container is object.

Q. Why should Docker images avoid latest tag?

Answer: latest is mutable and can point to different content over time, making builds and rollbacks non-reproducible. Use version tags or digests.

Trick / clue: latest is not a version.

Q. How do you reduce Docker image size?

Answer: Use multi-stage builds, minimal base images, remove caches, copy only needed files, use .dockerignore, and avoid installing unnecessary packages.

Trick / clue: Smaller image, faster pull, less risk.

Q. What is .dockerignore?

Answer: .dockerignore excludes files from Docker build context, reducing build time and preventing secrets or large files from being sent to the daemon.

Trick / clue: It is like .gitignore for image builds.

Q. What is container layer caching?

Answer: Docker reuses unchanged build layers. Put stable dependency steps before frequently changing source code to speed builds.

Trick / clue: Order Dockerfile from least-changing to most-changing.

Q. How do containers isolate processes?

Answer: Linux namespaces isolate process, network, mount, IPC, and user views; cgroups control resource usage.

Trick / clue: Namespaces hide; cgroups limit.

Q. What is privileged container risk?

Answer: Privileged containers get broad host capabilities and can compromise nodes. Avoid unless absolutely necessary and isolate heavily.

Trick / clue: Privileged means near-host access.

Q. How do you debug a container that exits immediately?

Answer: Check logs, entrypoint/cmd, environment, missing files, permissions, dependency connections, architecture mismatch, and run interactively with overridden command.

Trick / clue: If it exits, inspect command and logs.

Q. What is a distroless image?

Answer: A distroless image contains app runtime dependencies without package manager or shell, reducing attack surface but making debugging harder.

Trick / clue: Distroless = secure minimal runtime.

Q. What is container image scanning?

Answer: Scanning checks image packages/libraries for known vulnerabilities. Strong pipelines scan but also prioritize based on exploitability and runtime exposure.

Trick / clue: Scan, prioritize, patch.

Q. How do you handle secrets in containers?

Answer: Do not bake secrets into images. Inject via secret manager, Kubernetes secrets, mounted volumes, or runtime environment with least privilege and rotation.

Trick / clue: Images are forever; secrets must rotate.

Q. Why use non-root containers?

Answer: Non-root reduces impact if an attacker compromises the application, especially when combined with dropped capabilities and read-only filesystem.

Trick / clue: Non-root limits blast radius.

Kubernetes Core

Q. Explain control plane components.

Answer: API server handles requests, etcd stores state, scheduler places Pods, controller-manager reconciles desired state, and cloud-controller integrates cloud resources.

Trick / clue: API server front door; etcd memory; scheduler places; controllers reconcile.

Q. What is etcd?

Answer: etcd is Kubernetes distributed key-value store holding cluster state. It must be backed up and protected because cluster recovery depends on it.

Trick / clue: etcd is cluster brain storage.

Q. What happens when you create a Deployment?

Answer: kubectl sends manifest to API server, it is stored in etcd, Deployment controller creates ReplicaSet, ReplicaSet creates Pods, scheduler assigns nodes, kubelet pulls images and starts containers.

Trick / clue: API -> etcd -> controllers -> scheduler -> kubelet.

Q. What is kubelet?

Answer: kubelet runs on each node and ensures containers described by assigned Pods are running and healthy.

Trick / clue: kubelet is node agent.

Q. What is kube-proxy?

Answer: kube-proxy implements Service networking rules on nodes using iptables/ipvs or equivalent mechanisms.

Trick / clue: kube-proxy maps Service IP to Pod endpoints.

Q. What is an EndpointSlice?

Answer: EndpointSlice stores scalable lists of network endpoints backing Services, replacing older Endpoints limitations for large services.

Trick / clue: EndpointSlice = scalable service backend list.

Q. Why is a Pod not ready?

Answer: Readiness probe failure, container not started, app dependency not ready, wrong port/path, slow boot, or resource pressure.

Trick / clue: Ready controls traffic.

Q. What is a PVC?

Answer: PersistentVolumeClaim requests storage. Kubernetes binds it to a PersistentVolume provided statically or dynamically by a StorageClass.

Trick / clue: PVC asks for storage; PV provides it.

Q. What is StorageClass?

Answer: StorageClass defines dynamic storage provisioning parameters such as volume type, reclaim policy, encryption, and expansion support.

Trick / clue: StorageClass = storage template.

Q. What is ConfigMap checksum rollout?

Answer: A checksum annotation of config content on Pod template changes when config changes, causing Deployment rollout.

Trick / clue: Change pod template to restart pods.

Q. What is finalizer?

Answer: A finalizer blocks deletion until a controller performs cleanup. Stuck finalizers can leave resources terminating.

Trick / clue: Finalizer says clean up before delete.

Q. How do labels and selectors work?

Answer: Labels are key-value metadata. Selectors match labels to connect Deployments, Services, NetworkPolicies, and other controllers to resources.

Trick / clue: Labels glue Kubernetes objects together.

EKS and AWS Kubernetes

Q. What EKS components does AWS manage?

Answer: AWS manages the Kubernetes control plane availability, patching, and scaling. Users manage worker nodes, add-ons, IAM, networking, workloads, and cluster configuration.

Trick / clue: Managed control plane does not mean managed platform.

Q. What is aws-auth ConfigMap?

Answer: aws-auth maps AWS IAM roles/users to Kubernetes usernames/groups in older EKS access model. Newer access entries can replace parts of this workflow.

Trick / clue: aws-auth bridges IAM to Kubernetes RBAC.

Q. What is EKS managed node group?

Answer: Managed node groups automate lifecycle of EC2 worker nodes for EKS, including provisioning, updates, and health integration.

Trick / clue: Managed nodes reduce operational work.

Q. What is Fargate for EKS?

Answer: Fargate runs Pods without managing EC2 nodes, useful for isolated or bursty workloads but with limitations around DaemonSets, privileged workloads, and cost.

Trick / clue: Fargate = serverless pods.

Q. How does AWS VPC CNI affect IP usage?

Answer: Pods get VPC IP addresses, consuming subnet IPs. Large clusters require subnet sizing, prefix delegation, or alternate CNI planning.

Trick / clue: In EKS, pod IPs can exhaust subnets.

Q. What is prefix delegation?

Answer: Prefix delegation assigns IP prefixes to nodes to improve pod IP scalability and reduce ENI/IP management overhead.

Trick / clue: More pod IPs per node more efficiently.

Q. How do you expose services in EKS?

Answer: Use Service type LoadBalancer for NLB, Ingress with AWS Load Balancer Controller for ALB, or internal-only services with ClusterIP/private LB.

Trick / clue: Expose based on L4/L7 and internal/external needs.

Q. What is ECR?

Answer: Elastic Container Registry stores Docker/OCI images with IAM-based access, scanning, lifecycle policies, and regional replication options.

Trick / clue: ECR = AWS image registry.

Q. How do Pods access AWS services securely?

Answer: Use IRSA or EKS Pod Identity to assign IAM roles to service accounts/pods rather than using node instance role credentials.

Trick / clue: Pod-level IAM beats node-wide IAM.

Q. What is Karpenter consolidation?

Answer: Karpenter can replace or remove underutilized nodes to reduce cost while respecting Pod disruption constraints.

Trick / clue: Consolidation removes waste.

Q. How do you monitor EKS cluster health?

Answer: Monitor control plane logs, node health, pod restarts, pending pods, API errors, CoreDNS, CNI IP usage, add-on health, and SLOs.

Trick / clue: Cluster health = control plane + nodes + add-ons + apps.

Q. How do you handle EKS add-ons?

Answer: Version and manage add-ons such as VPC CNI, CoreDNS, kube-proxy, EBS CSI, EFS CSI, and external-dns through IaC/GitOps with compatibility checks.

Trick / clue: Add-ons are part of the platform, not afterthoughts.

Helm and Packaging

Q. What is Helm release history?

Answer: Helm stores release revisions, allowing inspection and rollback. Too much history can be limited with --history-max.

Trick / clue: History enables rollback.

Q. What is helm template used for?

Answer: helm template renders manifests locally without installing, useful for debugging, CI validation, and Argo CD review.

Trick / clue: Render before deploy.

Q. What is helm lint?

Answer: helm lint checks chart structure and common errors before deployment.

Trick / clue: Lint catches chart mistakes early.

Q. How do you validate Helm values?

Answer: Use values.schema.json to enforce types, required fields, enums, and constraints.

Trick / clue: Schema prevents bad values.

Q. What are Helm hooks?

Answer: Helm hooks run resources at lifecycle points like pre-install or post-upgrade. Use carefully because hooks can complicate GitOps and rollback.

Trick / clue: Hooks add lifecycle side effects.

Q. Why avoid too much logic in Helm templates?

Answer: Complex templates become hard to test and debug. Prefer simple templates, helper functions, schemas, and clear values.

Trick / clue: Templates should not become programs.

Q. How do you manage chart dependencies?

Answer: Use Chart.yaml dependencies, helm dependency update, version pinning, and repository governance.

Trick / clue: Pin dependencies for reproducibility.

Q. What is an umbrella chart?

Answer: An umbrella chart packages multiple subcharts. It can simplify deployment but may tightly couple independently released services.

Trick / clue: Umbrella is convenient but can reduce independence.

Q. How do you handle environment-specific Helm values?

Answer: Use separate values files or overlays per environment, keep secrets external, and promote image digests through PRs.

Trick / clue: Only env differences belong in env values.

Q. How do you debug wrong rendered values?

Answer: Run helm template with the exact values files and --debug, inspect final manifests, and compare with live kubectl/Argo diff.

Trick / clue: Rendered manifest is truth.

Q. What is release name impact?

Answer: Release name often influences Kubernetes object names through templates. Changing it can create new resources instead of updating existing ones.

Trick / clue: Name changes can look like reinstall.

Q. How do you design chart labels?

Answer: Use recommended Kubernetes labels: app.kubernetes.io/name, instance, version, component, part-of, managed-by for selectors and observability.

Trick / clue: Labels support ownership and queries.

Argo CD and GitOps Advanced

Q. What is sync wave vs hook difference?

Answer: Sync waves order normal resources. Hooks run special resources at lifecycle phases and may be deleted based on hook policies.

Trick / clue: Waves order; hooks act.

Q. What is self-heal risk?

Answer: Self-heal reverts manual cluster changes to Git state. This is good for drift but bad if emergency hotfix is not committed back to Git.

Trick / clue: Self-heal enforces Git even during panic.

Q. What is prune risk?

Answer: Prune deletes live resources removed from Git. It can delete critical resources if repo structure or generator output changes unexpectedly.

Trick / clue: Prune needs review.

Q. How do Argo CD Projects support multi-tenancy?

Answer: Projects restrict repositories, destinations, resource kinds, and roles so teams cannot deploy anywhere or use forbidden resources.

Trick / clue: Project is Argo authorization boundary.

Q. What is resource customization in Argo CD?

Answer: Resource customization changes diffing or health assessment for specific resource kinds, useful for controllers that mutate fields.

Trick / clue: Customize when default diff/health is noisy.

Q. How do you handle generated fields in Argo diff?

Answer: Use ignoreDifferences for fields managed by controllers, but avoid hiding real drift. Scope ignores narrowly.

Trick / clue: Ignore only controller-owned noise.

Q. What is app-of-apps risk?

Answer: A root app can impact many child apps. Protect it with review, manual sync for prod, and limited permissions.

Trick / clue: Root app has huge blast radius.

Q. How do you bootstrap a cluster with GitOps?

Answer: Install Argo CD, connect repos/clusters, apply root platform app, sync namespaces/add-ons, then app teams onboard through Application/ApplicationSet.

Trick / clue: Bootstrap once, reconcile forever.

Q. How do you roll back with GitOps?

Answer: Revert the Git commit or change image/chart version back, merge PR, and sync. Argo history rollback can work but Git revert preserves source of truth.

Trick / clue: Rollback through Git keeps audit.

Q. What are sync windows?

Answer: Sync windows restrict when apps can sync, useful for prod change windows or freeze periods.

Trick / clue: Time-based deployment guardrail.

Q. What is orphaned resource monitoring?

Answer: Argo CD can detect resources in a namespace not managed by an Application, helping find manual drift or abandoned resources.

Trick / clue: Orphans reveal unmanaged resources.

Q. How do you monitor Argo CD?

Answer: Track app sync/health status, reconciliation errors, repo server failures, controller queue, Redis health, API latency, and failed sync notifications.

Trick / clue: GitOps controller needs SLOs too.

Terraform and Infrastructure as Code

Q. What is terraform import?

Answer: Import maps an existing real resource into Terraform state so Terraform can manage it. You still need matching configuration.

Trick / clue: Import state, then write config.

Q. What is terraform taint or replace?

Answer: Forces Terraform to destroy and recreate a resource. Modern usage prefers terraform apply -replace=resource.

Trick / clue: Replace recreates deliberately.

Q. What causes Terraform state drift?

Answer: Manual console changes, external automation, provider defaults, deleted resources, or configuration not matching reality.

Trick / clue: Drift = state/config/reality mismatch.

Q. How do you structure Terraform for environments?

Answer: Use reusable modules and separate live environment directories or stacks with separate state per account/region/env.

Trick / clue: Separate state by blast radius.

Q. What is provider version pinning?

Answer: Pin provider versions to avoid unexpected behavior changes during init or CI runs.

Trick / clue: Pin for reproducible plans.

Q. What is a data source?

Answer: A data source reads existing provider information without managing it, such as AMI IDs, VPC IDs, or IAM policy documents.

Trick / clue: Data source reads; resource manages.

Q. What is lifecycle ignore_changes?

Answer: It tells Terraform to ignore changes to selected attributes, useful for externally managed fields but dangerous if overused.

Trick / clue: Ignore only intentional external changes.

Q. What is create_before_destroy?

Answer: Lifecycle rule that creates replacement before destroying old resource to reduce downtime, if names/capacity allow.

Trick / clue: Create first, cut over, destroy old.

Q. How do you handle Terraform secrets?

Answer: Avoid putting secrets in variables/state. Use secret managers, sensitive variables, encrypted state, and limit state access.

Trick / clue: State can contain secrets.

Q. What is policy check for Terraform?

Answer: Use tools like Sentinel, OPA/Conftest, Checkov, tfsec, or Terrascan to enforce security/cost/compliance before apply.

Trick / clue: Policy catches bad infra before cloud does.

Q. How do you handle failed Terraform apply halfway?

Answer: Inspect state and real resources, rerun plan, avoid manual deletion unless necessary, import or taint resources if needed, and restore consistency.

Trick / clue: Bring state and reality back together.

Q. Why avoid huge monolithic Terraform state?

Answer: Large state increases blast radius, slows plans, complicates locking, and makes ownership unclear. Split by lifecycle and ownership.

Trick / clue: Small states reduce blast radius.

CloudBees Jenkins and Enterprise CI

Q. What is Jenkinsfile?

Answer: Jenkinsfile stores pipeline definition as code, usually declarative or scripted pipeline, enabling review and versioning.

Trick / clue: Jenkinsfile = pipeline as code.

Q. Declarative vs scripted pipeline?

Answer: Declarative has structured syntax and guardrails. Scripted is more flexible Groovy but harder to govern.

Trick / clue: Declarative for standards, scripted for flexibility.

Q. What is Jenkins shared library?

Answer: Reusable pipeline code stored centrally and loaded by Jenkinsfiles to standardize CI/CD logic.

Trick / clue: Shared library = common pipeline functions.

Q. How do you secure Jenkins credentials?

Answer: Store in credentials manager, scope to folders/jobs, avoid echoing, rotate regularly, restrict admin access, and audit usage.

Trick / clue: Credentials are powerful; scope them.

Q. How do you manage Jenkins plugins safely?

Answer: Pin/test plugin versions, update regularly, remove unused plugins, monitor advisories, and test in non-prod controller.

Trick / clue: Plugins are supply-chain risk.

Q. How do you scale Jenkins builds?

Answer: Use distributed agents, autoscaling cloud agents, labels, queue monitoring, pipeline parallelism, and avoid building on controller.

Trick / clue: Controller schedules, agents build.

Q. What is workspace cleanup?

Answer: Cleaning workspaces prevents stale files, secret leakage, disk pressure, and nondeterministic builds.

Trick / clue: Dirty workspace causes weird builds.

Q. How do you handle flaky tests in CI?

Answer: Track flake rate, quarantine carefully, retry only with reporting, fix root cause, and avoid hiding real failures.

Trick / clue: Retries should expose flakes, not bury them.

Q. What is CloudBees CasC?

Answer: Configuration as Code manages Jenkins/CloudBees configuration declaratively, improving repeatability and auditability.

Trick / clue: Controller config should be code too.

Q. How do you handle pipeline approvals?

Answer: Use environment gates, input steps sparingly, RBAC, change records, and audit logs for production deployments.

Trick / clue: Approvals protect prod but can become bottlenecks.

Q. How do you archive artifacts?

Answer: Archive only necessary outputs with retention policies. Prefer artifact repositories for large/versioned artifacts.

Trick / clue: Do not turn Jenkins into long-term storage.

Q. What is build promotion?

Answer: Promoting a tested artifact across environments avoids rebuilding and ensures prod runs what was tested.

Trick / clue: Promote artifacts, not source.

Security and Compliance

Q. What is zero trust?

Answer: Zero trust assumes no implicit trust based on network location. Every request should be authenticated, authorized, encrypted, and monitored.

Trick / clue: Never trust, always verify.

Q. What is least privilege in Kubernetes?

Answer: Grant service accounts/users only required verbs on required resources in required namespaces.

Trick / clue: No cluster-admin by default.

Q. What is secrets sprawl?

Answer: Secrets spread across repos, CI variables, local files, and clusters. Reduce by centralizing in secret manager and rotating.

Trick / clue: Secrets multiply unless governed.

Q. How do you respond to leaked AWS keys?

Answer: Disable/revoke key, rotate affected secrets, audit CloudTrail usage, identify blast radius, redeploy with new credentials, and add prevention scanning.

Trick / clue: Revoke first, investigate next.

Q. What is runtime security?

Answer: Monitoring and controlling behavior of running workloads, such as unexpected processes, network calls, file writes, or privilege escalation.

Trick / clue: Build-time scan is not enough.

Q. How do you enforce image provenance?

Answer: Require signed images from trusted CI, verify signatures at admission, and block unsigned/manual images.

Trick / clue: Only trusted pipeline can produce deployable images.

Q. What is WAF used for?

Answer: Web Application Firewall filters malicious HTTP traffic like SQL injection, XSS, known bad IPs, and bot patterns before reaching apps.

Trick / clue: WAF is HTTP shield, not app security replacement.

Q. What is mTLS?

Answer: Mutual TLS authenticates both client and server with certificates, often used for service-to-service identity.

Trick / clue: Both sides show certificates.

Q. What is audit logging?

Answer: Audit logs record who did what, when, and from where. They are essential for incident investigation and compliance.

Trick / clue: Audit logs answer accountability.

Q. How do you secure Kubernetes API server access?

Answer: Private endpoint, RBAC, MFA through identity provider, audit logs, least privilege, network restrictions, and no long-lived admin kubeconfigs.

Trick / clue: API server is cluster front door.

Q. What is CVSS limitation?

Answer: CVSS rates severity but not exploitability in your environment. Prioritize by exposure, asset criticality, exploit availability, and compensating controls.

Trick / clue: Severity is not the same as risk.

Q. How do you handle compliance without slowing delivery?

Answer: Automate controls as policy-as-code, standard templates, evidence collection, CI gates, and self-service guardrails.

Trick / clue: Compliance should be paved road, not paperwork.

Observability and SRE

Q. What is SLI?

Answer: Service Level Indicator is a specific measurement of service behavior, such as successful request rate or latency below threshold.

Trick / clue: SLI is the measurement.

Q. What is SLA?

Answer: Service Level Agreement is an external contract with consequences if missed.

Trick / clue: SLA is business/legal promise.

Q. SLO vs SLA?

Answer: SLO is internal reliability target; SLA is external contractual commitment. SLO should usually be stricter than SLA.

Trick / clue: Objective internal, agreement external.

Q. What is cardinality?

Answer: Number of unique time series produced by label combinations. High cardinality increases monitoring cost and query latency.

Trick / clue: Every label multiplies series.

Q. What are exemplars?

Answer: Exemplars link metrics to trace samples, helping jump from a latency spike to example traces.

Trick / clue: Metric spike to trace detail.

Q. What is distributed tracing sampling?

Answer: Sampling chooses which traces to keep due to cost. Tail-based sampling can keep interesting slow/error traces.

Trick / clue: Keep traces that teach something.

Q. What is saturation?

Answer: How full or overloaded a resource is, such as CPU, memory, disk, queue length, thread pool, or connection pool.

Trick / clue: Saturation means near limit.

Q. What is synthetic monitoring?

Answer: Automated probes simulate user actions to test availability and latency from outside or inside networks.

Trick / clue: Robot user checks service.

Q. What is black-box vs white-box monitoring?

Answer: Black-box observes from outside like a user. White-box uses internal metrics/logs from the system.

Trick / clue: Outside symptoms plus inside causes.

Q. How do you detect dependency failures?

Answer: Use traces, dependency dashboards, client-side error metrics, circuit breaker metrics, and upstream health checks.

Trick / clue: Dependencies fail through your app.

Q. What is alert fatigue?

Answer: Too many low-value alerts cause responders to ignore or miss important pages. Fix by making alerts actionable and SLO-based.

Trick / clue: Noise kills response.

Q. How do you measure toil?

Answer: Track repetitive manual operational tasks by time, frequency, interrupt cost, and automation opportunity.

Trick / clue: Measure toil to justify automation.

Company-Style Scenario Prompts

Q. Meta-style scale prompt: CI jobs increased 5x and queues are slow. What do you do?

Answer: Measure queue vs execution time, shard runners, autoscale ephemeral runners, optimize caching, split tests, reduce redundant workflows, prioritize critical jobs, and set concurrency controls.

Trick / clue: Scale bottlenecks require metrics first.

Q. Cisco-style enterprise prompt: migrate on-prem platform workloads to EKS.

Answer: Inventory parity, map network/security/storage/secrets, create AWS landing zone, deploy platform add-ons, validate non-prod, dual-run, cut over, monitor, and decommission.

Trick / clue: Enterprise migration values parity and governance.

Q. Palo Alto/Zscaler-style security prompt: prevent risky Kubernetes deployments.

Answer: Use admission policies, image signing, vulnerability gates, RBAC, network policy, pod security standards, secrets controls, and audit logs.

Trick / clue: Security companies expect preventive controls.

Q. Zoom-style reliability prompt: deployment causes latency during peak traffic.

Answer: Rollback or canary pause, compare version metrics, inspect CPU throttling/GC/downstream latency, protect user experience, then root cause after mitigation.

Trick / clue: During user impact, mitigate before deep analysis.

Q. Oracle-style database prompt: app deployment needs schema change.

Answer: Use expand/contract migration, backward-compatible code, migration jobs, backups, validation queries, and rollback/forward-fix plan.

Trick / clue: Data changes need backward compatibility.

Q. Startup prompt: limited budget, need production platform fast.

Answer: Start simple: managed services, GitHub Actions, IaC, containers, one EKS or ECS platform, basic observability, backups, secrets, and documentation. Avoid complex service mesh unless needed.

Trick / clue: Startups need speed plus sane guardrails.

Q. FAANG-style troubleshooting prompt: random p99 spikes.

Answer: Correlate p99 with deploys, GC, CPU throttling, noisy neighbors, dependency latency, lock contention, retries, and network. Use traces/profiles and isolate by endpoint/version/node.

Trick / clue: Tail latency hides in rare slow paths.

Q. FAANG-style design prompt: multi-region service deployment.

Answer: Discuss active-active/passive, data consistency, traffic routing, failover, observability, deployment strategy, and cost. Define RTO/RPO.

Trick / clue: Multi-region is mostly data and operations.

Q. AppZen-style SaaS prompt: customer data security in CI/CD.

Answer: Separate tenants/environments, avoid prod data in CI, encrypt data, audit access, scan artifacts, isolate runners, and enforce least privilege.

Trick / clue: CI/CD should never casually touch customer data.

Q. Bay Area startup prompt: first SRE hire priorities.

Answer: Establish incident process, monitoring, CI/CD reliability, backups, infrastructure as code, security basics, cost visibility, and developer self-service.

Trick / clue: First SRE builds leverage.

Q. Senior prompt: how do you choose between Argo CD and Jenkins deploy?

Answer: Jenkins is pipeline orchestrator; Argo CD is Kubernetes GitOps reconciler. Use Jenkins/GitHub for build/test/package and Argo for cluster deployment/drift control.

Trick / clue: CI builds; GitOps deploys.

Q. Staff prompt: reduce platform fragmentation across teams.

Answer: Create golden paths, reusable modules/charts, governance, migration support, docs, metrics, and exceptions process. Influence through value, not mandates only.

Trick / clue: Standardization succeeds when it helps teams move faster.

6. Deep Coverage: Git, GitHub Actions, Runners, CloudBees/Jenkins

These questions are common in DevOps interviews because CI/CD systems are production systems. Strong answers must include security, repeatability, rollback, and auditability.

Q. Explain a production-grade Git branching strategy.

Answer: Use trunk-based development for fast teams: short-lived branches, PR review, required checks, feature flags, and release tags. GitFlow may fit slower release cycles but can create long-lived branch drift. For platform teams, protect main, require signed commits where appropriate, and promote artifacts across environments rather than rebuilding.

Trick / clue: Trunk-based = small changes + strong automation.

Q. How do you design reusable GitHub Actions workflows?

Answer: Create workflow_call reusable workflows for common build, test, scan, and deploy logic. Standardize inputs/outputs, secrets usage, permissions, concurrency, artifact handling, and version them by tags or SHA.

Trick / clue: Reusable workflows reduce copy-paste pipeline drift.

Q. What permissions should GitHub Actions workflows use?

Answer: Set default permissions to read-only and grant minimum job-specific permissions like contents:read, id-token:write for OIDC, packages:write only for publishing, and pull-requests:write only if needed.

Trick / clue: permissions: {} first, then add what is required.

Q. How do you avoid running dangerous workflows on forked PRs?

Answer: Do not expose secrets to untrusted pull_request workflows. Use pull_request_target carefully because it runs in base context. Prefer isolated checks without secrets and require maintainer approval for external contributors.

Trick / clue: Fork PRs are untrusted code.

Q. How do you design CloudBees shared libraries?

Answer: Put common pipeline functions into versioned shared libraries, enforce code review, provide stable APIs, document usage, and test libraries like production code. Avoid hiding too much magic from app teams.

Trick / clue: Shared library = reusable pipeline product.

Q. How do you compare Jenkins pipeline and GitHub Actions?

Answer: Jenkins/CloudBees is flexible and enterprise-customizable, good for complex internal networks and legacy integrations. GitHub Actions is repository-native, easier for GitHub-centric teams, and integrates well with PR checks and OIDC. Both need runner/agent security and governance.

Trick / clue: Tool choice depends on ecosystem, governance, and operations burden.

Q. What is pipeline as code?

Answer: Pipeline as code stores build and deployment definitions in version control so changes are reviewed, versioned, reproducible, and auditable.

Trick / clue: Pipeline should be code-reviewed like app code.

Q. How do you handle secrets in CI/CD?

Answer: Prefer OIDC and short-lived credentials. Store secrets in managed secret stores, restrict environment access, mask logs, avoid echoing secrets, rotate regularly, and never put secrets in repo or artifacts.

Trick / clue: Secrets leak through logs, artifacts, caches, and runners.

Q. What is a concurrency group in GitHub Actions?

Answer: Concurrency prevents overlapping workflow runs for the same branch/environment. It can cancel in-progress runs to avoid stale deployments.

Trick / clue: Concurrency prevents two deploys racing.

Q. How do you debug a stuck Jenkins queue?

Answer: Check executor availability, node labels, offline agents, throttling, locks, quiet period, resource limits, and controller logs. Confirm jobs are not waiting for unavailable labels or approvals.

Trick / clue: Queue stuck usually means no matching healthy agent.

7. Deep Coverage: Docker, Helm, Kubernetes, EKS

Q. What makes a good Docker image?

Answer: Small, secure, reproducible, non-root, minimal packages, pinned versions, multi-stage build, healthcheck where useful, no secrets, correct entrypoint, and vulnerability scanning.

Trick / clue: Small image = smaller attack surface and faster deploys.

Q. Explain multi-stage Docker builds.

Answer: Build dependencies in one stage and copy only runtime artifacts into a smaller final image. This reduces size and avoids shipping compilers or build secrets.

Trick / clue: Build big, run small.

Q. How do you make containers run as non-root?

Answer: Set USER in Dockerfile, configure Kubernetes securityContext runAsNonRoot/runAsUser, ensure file permissions allow app writes only where needed, and avoid privileged capabilities.

Trick / clue: Non-root reduces container escape impact.

Q. What is a sidecar?

Answer: A sidecar is a helper container in the same Pod that supports the main app, such as log shipper, proxy, service mesh, or config reloader.

Trick / clue: Sidecar shares pod network and lifecycle.

Q. What is an init container?

Answer: An init container runs before app containers to perform setup such as migrations, permission setup, or dependency checks.

Trick / clue: Init must finish before app starts.

Q. How do you debug Kubernetes service routing?

Answer: Check Service selector labels, Endpoints/EndpointSlices, pod readiness, targetPort vs containerPort, kube-proxy/CNI, NetworkPolicy, and DNS resolution.

Trick / clue: Service without endpoints sends traffic nowhere.

Q. What is CNI?

Answer: Container Network Interface plugin provides pod networking. In EKS, the AWS VPC CNI assigns VPC IPs to Pods unless alternate CNI is used.

Trick / clue: CNI gives pods network identity.

Q. How do you handle private EKS clusters?

Answer: Use private API endpoint, VPN/bastion/SSM or VPC connectivity for admins and CI, private subnets for nodes, VPC endpoints for AWS services, and controlled ingress/egress.

Trick / clue: Private cluster needs private access path.

Q. How do you configure EKS access control?

Answer: Use AWS IAM authentication mapped to Kubernetes RBAC through EKS access entries or aws-auth ConfigMap depending on setup. Grant least privilege Roles and avoid cluster-admin except break-glass.

Trick / clue: AWS authenticates; Kubernetes authorizes.

Q. What are taints and tolerations?

Answer: Taints repel Pods from nodes unless Pods have matching tolerations. They are used for dedicated, GPU, spot, or special-purpose nodes.

Trick / clue: Taint says keep out; toleration says I can enter.

Q. What is node affinity?

Answer: Node affinity attracts Pods to nodes matching labels. Required affinity is strict; preferred affinity is soft.

Trick / clue: Affinity attracts; taints repel.

Q. How do you design Helm values for multiple environments?

Answer: Keep common defaults in values.yaml and environment overlays for dev/stage/prod. Store secrets externally. Use schema validation and avoid duplicating full files when only small differences exist.

Trick / clue: Defaults plus overlays.

Q. How do you trigger pod rollout when ConfigMap changes?

Answer: Add checksum annotations on Pod template based on rendered ConfigMap/Secret contents, or use a reloader controller. Changing Pod template triggers a new ReplicaSet.

Trick / clue: Deployment restarts only when pod template changes.

Q. What is a StatefulSet?

Answer: StatefulSet manages stateful Pods with stable network identity, ordered deployment, and persistent volume claims per replica.

Trick / clue: StatefulSet = stable name + stable storage.

Q. What is a DaemonSet?

Answer: DaemonSet runs one Pod per matching node, useful for agents like log collectors, monitoring agents, or CNI components.

Trick / clue: DaemonSet = every node gets one.

8. Deep Coverage: Argo CD and GitOps

Q. Explain Argo CD sync waves.

Answer: Sync waves order resources during sync using `argocd.argoproj.io/sync-wave` annotation. Lower waves apply first. Use for namespaces, CRDs, operators, then app resources.

Trick / clue: Wave order: platform prerequisites before app.

Q. What are Argo CD hooks?

Answer: Hooks run jobs or resources at phases like PreSync, Sync, PostSync, SyncFail, or PostDelete. Use carefully for migrations or validations.

Trick / clue: Hooks are lifecycle actions around sync.

Q. How do you avoid accidental prune in Argo CD?

Answer: Use project restrictions, code review, disable prune by default for risky apps, require manual sync for prod, resource finalizers carefully, and app-level exclusions for resources managed elsewhere.

Trick / clue: Prune deletes what Git removed.

Q. What does Argo CD health mean?

Answer: Health is Argo CD interpretation of resource readiness. Synced means desired/live match; Healthy means resource reports ready. An app can be Synced but Unhealthy.

Trick / clue: Sync = same config; health = actually working.

Q. How do you structure GitOps repositories?

Answer: Common patterns are app repo plus environment config repo, mono-repo for platform, or app-of-apps. Use clear separation for base charts, env values, cluster add-ons, and secrets references.

Trick / clue: Repo structure should match ownership and promotion flow.

Q. What is app-of-apps pattern?

Answer: A root Argo CD Application manages child Applications, letting teams bootstrap many apps from Git. It is powerful but needs guardrails to avoid broad blast radius.

Trick / clue: App-of-apps = GitOps tree.

Q. How do you promote from dev to prod in GitOps?

Answer: Promote immutable image digests or chart versions via PR from dev values to stage/prod values, run checks, require approval, then Argo CD syncs prod.

Trick / clue: Promote config, not rebuilds.

Q. How do you manage secrets in GitOps?

Answer: Do not store plaintext secrets. Use External Secrets references, sealed secrets, SOPS with KMS, or secret store CSI. Ensure decryption/sync access is least privilege.

Trick / clue: Git stores references or encrypted values, not raw secrets.

Q. How do you handle emergency hotfixes with GitOps?

Answer: Prefer committing the hotfix to Git and syncing. If manual cluster change is unavoidable, document break-glass, then backport the change to Git immediately to prevent drift/self-heal reversal.

Trick / clue: Hotfix must return to Git source of truth.

Q. What is Argo CD Project?

Answer: A Project groups Applications and restricts source repositories, destination clusters/namespaces, resource kinds, and roles. It is a key multi-tenant security boundary.

Trick / clue: Project = Argo CD tenancy guardrail.

9. Deep Coverage: Terraform, AWS, IAM, Networking

Q. How do you design Terraform modules?

Answer: Modules should have clear inputs/outputs, sane defaults, versioning, examples, validation, minimal provider assumptions, no hardcoded environment values, and documentation.

Trick / clue: Module = reusable product, not copy-paste folder.

Q. What is the difference between security groups and NACLs?

Answer: Security groups are stateful instance/ENI-level firewalls. NACLs are stateless subnet-level controls. Security groups are commonly primary; NACLs are coarse subnet guardrails.

Trick / clue: SG stateful close; NACL stateless subnet.

Q. What is NAT Gateway used for?

Answer: NAT Gateway lets resources in private subnets initiate outbound internet access without allowing inbound internet connections.

Trick / clue: Private subnet outbound path.

Q. Explain public vs private subnet.

Answer: A public subnet has a route to an internet gateway and can host internet-facing resources. A private subnet lacks direct inbound internet route and usually uses NAT for outbound.

Trick / clue: Public has IGW route; private does not.

Q. How do you design IAM policies?

Answer: Start with least privilege actions/resources, use conditions, separate roles by workload, use temporary credentials, avoid wildcards, and test with IAM Access Analyzer or policy simulator.

Trick / clue: Action + resource + condition.

Q. How does EKS use IAM and RBAC together?

Answer: IAM authenticates AWS identity to cluster, then Kubernetes RBAC authorizes actions on Kubernetes resources. IRSA gives pods IAM roles for AWS APIs.

Trick / clue: Humans/nodes/pods need different identities.

Q. What is AWS Secrets Manager vs Parameter Store?

Answer: Secrets Manager is designed for secrets with rotation and secret lifecycle. Parameter Store can store config and secrets, often cheaper/simple, with SecureString via KMS.

Trick / clue: Secrets Manager for rotation; Parameter Store for config/simple secret values.

Q. What is an AWS Load Balancer target group?

Answer: A target group is a set of targets such as instances or IPs that receive traffic from ALB/NLB listeners based on health checks and routing rules.

Trick / clue: LB listener routes to target group.

Q. How do you troubleshoot TLS certificate issues?

Answer: Check certificate chain, expiration, SAN names, issuer, private key match, ingress annotations, listener config, DNS, and client trust. Use `openssl s_client` for inspection.

Trick / clue: TLS failures often = name, chain, expiry, or wrong cert.

Q. How would you design private connectivity to AWS services?

Answer: Use VPC endpoints: gateway endpoints for S3/DynamoDB, interface endpoints for services like ECR, STS, Secrets Manager, CloudWatch, and private DNS.

Trick / clue: Endpoints keep traffic inside AWS network.

10. Deep Coverage: Observability, SRE, Incident Response

Q. What should be in a service dashboard?

Answer: Golden signals, deployment version, dependency health, saturation, error budget burn, traffic by route, p95/p99 latency, pod restarts, CPU/memory, and recent incidents.

Trick / clue: Dashboard should answer: is it broken, where, and since when?

Q. What is high-cardinality metric risk?

Answer: Too many unique label combinations increase cost and query load. Avoid labels like `user_id`, `request_id`, or unbounded path values.

Trick / clue: Labels multiply time series.

Q. How do you create a runbook?

Answer: Include alert meaning, impact, dashboards, commands, dependency checks, mitigation steps, rollback steps, escalation contacts, and validation criteria.

Trick / clue: Runbook = what to check and what to do.

Q. How do you manage on-call handoff?

Answer: Share active incidents, risky changes, silences, dashboards, known issues, pending actions, and escalation expectations.

Trick / clue: Handoff prevents context loss.

Q. How do you calculate availability?

Answer: Availability is successful time or requests divided by total. For request-based SLO, availability = good events / total events.

Trick / clue: Use events for APIs, time for infrastructure.

Q. What is burn rate alerting?

Answer: Burn rate measures how quickly error budget is being consumed. Multi-window burn alerts detect fast outages and slow reliability degradation.

Trick / clue: Burn rate tells how fast budget is burning.

Q. How do you reduce MTTR?

Answer: Improve detection, dashboards, ownership, runbooks, rollback automation, feature flags, incident command, and postmortem corrective actions.

Trick / clue: MTTR = detect + diagnose + mitigate.

Q. What is root cause vs contributing factor?

Answer: Root cause is often not a single thing. Strong postmortems identify technical and organizational contributing factors that allowed the incident to happen or last longer.

Trick / clue: Avoid one-root-cause oversimplification.

Q. How do you handle incident communication?

Answer: Use an incident commander, regular updates, single source of truth, severity levels, customer impact language, and clear next update time.

Trick / clue: During incidents, silence creates anxiety.

Q. What is graceful degradation?

Answer: A system continues offering core functionality when dependencies fail, by disabling non-critical features, using caches, or fallback responses.

Trick / clue: Fail soft, not total.

11. Security, Secrets, Supply Chain, Compliance

Q. What security checks belong in CI/CD?

Answer: Secret scanning, dependency scanning, SAST, IaC scanning, container image scanning, license checks, SBOM generation, signature/provenance, and policy gates based on severity.

Trick / clue: Shift left, but do not block without clear policy.

Q. What is SBOM?

Answer: Software Bill of Materials lists components, dependencies, and versions in an artifact, helping vulnerability management and compliance.

Trick / clue: SBOM = ingredients label for software.

Q. What is image signing?

Answer: Image signing cryptographically proves an image came from a trusted build process and was not tampered with. Admission policies can require valid signatures.

Trick / clue: Sign what you run.

Q. What is SLSA?

Answer: SLSA is a supply-chain security framework describing levels of build integrity, provenance, and tamper resistance.

Trick / clue: SLSA = maturity levels for trusted builds.

Q. How do you prevent secrets from being committed?

Answer: Use pre-commit hooks, secret scanning in CI and repository platform, education, protected branches, and immediate rotation if leaked.

Trick / clue: Detection is not enough; rotate leaked secrets.

Q. How do you secure Kubernetes RBAC?

Answer: Use least privilege Roles, namespace-scoped access, avoid cluster-admin, bind groups not individuals, review permissions, and audit access.

Trick / clue: RBAC sprawl becomes hidden admin access.

Q. What is Pod Security Admission?

Answer: Pod Security Admission enforces Kubernetes pod security standards such as privileged, baseline, and restricted at namespace level.

Trick / clue: PSA blocks unsafe pod specs.

Q. How do you protect against container privilege escalation?

Answer: Run as non-root, drop capabilities, read-only root filesystem, seccomp/AppArmor, no privileged mode, no hostPath unless required, and restrict service account tokens.

Trick / clue: Containers are not security boundaries by default.

Q. What is network egress control?

Answer: Egress control limits outbound traffic from workloads to approved destinations, reducing exfiltration and dependency sprawl.

Trick / clue: Not every pod should reach the internet.

Q. How do you handle vulnerability exceptions?

Answer: Document risk, owner, expiration date, compensating controls, affected assets, and remediation plan. Exceptions should be time-bound and reviewed.

Trick / clue: Exception without expiry becomes policy bypass.

12. Production Troubleshooting Workflows

Workflow: Kubernetes application is down

1. Confirm user impact and time window.
2. Check recent deployments and Argo CD sync history.
3. Check ingress/LB health and DNS.
4. Check Service endpoints and readiness.
5. Check pod status, restarts, events, and logs.
6. Check dependencies: DB, cache, queue, external APIs.
7. Mitigate: rollback, scale, restart, disable feature, or route traffic.
8. Validate recovery with metrics and synthetic checks.

Memory clue: Outside-in, then app-internal.

Workflow: Argo CD app is OutOfSync

1. Open diff and identify resource changes.
2. Check whether Git changed or live cluster drifted.
3. Review sync policy, prune, hooks, waves, and project restrictions.
4. Run dry-run or manual sync in lower environment.
5. Sync and watch health.
6. If drift was manual, backport correct state to Git.

Memory clue: Diff first. Do not blindly sync prod.

Workflow: CI/CD deployment failed

1. Find failed stage and exact log line.
2. Classify: code, dependency, test, credential, runner, network, deploy, or policy.
3. Reproduce locally if code/build issue.
4. Check secrets and cloud identity if auth issue.
5. Check runner health and capacity if infra issue.
6. Fix root cause and rerun from safe stage.
7. Add a test or guardrail to prevent recurrence.

Memory clue: Pipeline failures are data; classify before fixing.

Workflow: Terraform apply failed

1. Read provider error and resource address.
2. Check state lock and backend health.
3. Run terraform plan to confirm desired diff.
4. Check cloud console only for validation, not manual repair unless emergency.
5. Fix config/state/import issue.
6. Apply in lower env or targeted safe scope if needed.
7. Document drift or provider limitation.

Memory clue: Never panic-edit state unless you know exactly why.

Workflow: High CPU in production

1. Check if CPU is user, system, iowait, or throttling.
2. Identify pod/container/process.
3. Correlate with traffic and deployment.
4. Check HPA, requests/limits, node saturation.
5. Capture profiles if possible.
6. Mitigate by scaling or rollback.
7. Fix inefficient code, queries, retries, or resource sizing.

Memory clue: CPU high is symptom, not root cause.

Workflow: High memory / OOMKills

1. Identify container and OOM timestamp.
2. Check memory limits, working set, heap, caches, leaks.
3. Compare to previous version.
4. Check traffic spikes and batch jobs.

5. Mitigate by rollback, limit tuning, scaling, or feature flag.
6. Add heap dumps/profiling and memory alerts.

Memory clue: OOMKilled means container crossed limit; find why.

13. Scenario and Design Rounds

Q. Design a platform for 100 microservices on EKS.

Answer: Use multi-account AWS, shared networking, EKS clusters by environment, GitOps with Argo CD, standardized Helm charts, app templates, observability defaults, SLOs, secrets management, RBAC, network policies, autoscaling, cost controls, and self-service onboarding. Discuss tradeoffs: one large cluster vs many clusters, central vs team ownership, and blast radius.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. A deployment passed CI but failed in production. How do you investigate?

Answer: Compare environment differences, Helm values, secrets, feature flags, config, database schema, traffic pattern, resource limits, and dependency versions. Check Argo diff, deployment events, logs, metrics, traces, and rollback first if customer impact is active.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. Your company wants to move from Jenkins to GitHub Actions. What is your plan?

Answer: Inventory pipelines, classify complexity, create reusable workflows, design runners, secrets/OIDC, artifact registry, branch protections, migration waves, compatibility testing, training, and rollback. Keep Jenkins for special cases until parity is proven.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. Design a secure production release process.

Answer: Use PR reviews, required checks, artifact immutability, scans, approvals, GitOps promotion, progressive rollout, health gates, audit logs, rollback, and post-release monitoring.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. How would you onboard a new service to the platform?

Answer: Provide template repo, Dockerfile, Helm chart, CI workflow, Argo CD app, namespace/RBAC, service account/IRSA, secrets references, observability dashboard, alerts, runbook, and SLO definitions.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. Your Argo CD auto-sync deleted resources. What do you do?

Answer: Stop further sync/prune, assess impact, restore resources from Git/backup, identify commit causing deletion, revert or fix Git, validate app health, then add guardrails such as approvals, resource exclusions, or prune restrictions.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. A startup asks you to build DevOps from scratch. What comes first?

Answer: Start with version control, CI for tests/builds, container standard, cloud account/security baseline, IaC, basic Kubernetes or managed PaaS depending on team maturity, observability, secrets, backups, and incident process. Avoid overengineering.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. How would you design multi-region failover?

Answer: Define RTO/RPO, active-active or active-passive, data replication, DNS/traffic manager, regional isolation, secrets/IaC in both regions, health checks, runbooks, and regular failover tests.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. How do you handle a critical CVE in base images?

Answer: Identify affected images/services, prioritize internet-facing/high-risk, patch base image, rebuild artifacts, scan, promote through pipeline, deploy with change window if needed, verify, and create policy to prevent old base images.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

Q. How do you convince developers to adopt platform standards?

Answer: Offer paved-road templates that reduce work, explain reliability/security benefits, provide migration help, collect feedback, measure improved lead time/failure rate, and avoid blocking teams without support.

Trick / clue: For design rounds, always include tradeoffs, failure modes, and validation metrics.

14. Behavioral Questions and Strong Answers

Q. Tell me about a time you handled a production incident.

Answer: Use STAR. Situation: customer impact and severity. Task: your role. Action: triage, communication, mitigation, rollback/fix. Result: restored service and created follow-ups. Emphasize calm communication and blameless learning.

Trick / clue: Use STAR and end with measurable impact.

Q. Tell me about a time you disagreed with a developer or team.

Answer: Show respectful technical disagreement. Use data, logs, metrics, security/reliability risks, and propose options. Strong candidates do not escalate emotionally; they align on customer impact and tradeoffs.

Trick / clue: Use STAR and end with measurable impact.

Q. Tell me about a time you automated toil.

Answer: Describe manual repetitive work, risk/time cost, automation design, safety checks, adoption, and measurable time saved or error reduction.

Trick / clue: Use STAR and end with measurable impact.

Q. Tell me about a time you improved deployment reliability.

Answer: Mention failure rate, rollback time, test coverage, canary/blue-green, GitOps, health checks, and before/after metrics.

Trick / clue: Use STAR and end with measurable impact.

Q. Tell me about a time you learned a new technology quickly.

Answer: Explain how you built a small proof of concept, read docs, tested edge cases, documented learnings, and shared with team.

Trick / clue: Use STAR and end with measurable impact.

Q. How do you prioritize urgent requests from multiple teams?

Answer: Use impact, severity, deadlines, dependencies, and company goals. Communicate tradeoffs transparently and avoid silent context switching.

Trick / clue: Use STAR and end with measurable impact.

Q. What is your biggest DevOps weakness?

Answer: Pick a real but manageable weakness, such as wanting to go too deep before delegating. Explain the system you use to improve: timeboxing, documentation, peer review, and feedback.

Trick / clue: Use STAR and end with measurable impact.

Q. Why should we hire you for DevOps/SRE/platform engineering?

Answer: Connect hands-on delivery with reliability mindset: you can build pipelines/infrastructure, troubleshoot production, secure systems, document workflows, and improve developer velocity.

Trick / clue: Use STAR and end with measurable impact.

15. Cheat Sheets and Memory Tricks

DevOps master workflow cheat sheet

Topic	Workflow	Memory trick
CI/CD	Commit -> PR -> Test -> Build -> Scan -> Publish -> Deploy -> Verify	CPTBSPDV
Kubernetes debug	Events -> Status -> Logs -> Describe -> Exec -> Metrics	ESLDEM
Production incident	Detect -> Triage -> Mitigate -> Communicate -> Resolve -> Postmortem	DTMCRP
GitOps	Change Git -> Review -> Merge -> Reconcile -> Health -> Drift monitor	Git drives desired state
Terraform	Format -> Validate -> Plan -> Review -> Apply -> Output -> Drift check	FVPR AOD
Security	Identity -> Secrets -> Network -> Image -> Runtime -> Audit	ISNIRA
Observability	Metrics -> Logs -> Traces -> Events -> Profiles	MLTEP
Release	Build once -> Promote digest -> Canary -> Monitor -> Rollback if needed	Same artifact all the way

Command cheat sheet

Git

```
git status
git checkout -b feature/my-change
git add . && git commit -m "message"
git pull --rebase origin main
git log --oneline --graph --decorate --all
```

GitHub Actions

```
gh workflow list
gh run list
gh run view <run-id> --log
gh workflow run <workflow.yml>
```

Docker

```
docker build -t app:local .
docker run --rm -p 8080:8080 app:local
docker history app:local
docker scan app:local # or use trivy/grype in real pipelines
```

Kubernetes

```
kubectl get pods -n <ns> -o wide
kubectl describe pod <pod> -n <ns>
kubectl logs <pod> -n <ns> --previous
kubectl rollout status deploy/<name> -n <ns>
kubectl rollout undo deploy/<name> -n <ns>
kubectl get events -n <ns> --sort-by=.lastTimestamp
kubectl exec -it <pod> -n <ns> -- sh
```

Helm

```
helm lint ./chart
helm template app ./chart -f values-dev.yaml
helm upgrade --install app ./chart -n <ns> -f values-prod.yaml
helm history app -n <ns>
helm rollback app <revision> -n <ns>
```

Argo CD

```
argocd app list
argocd app get <app>
argocd app diff <app>
argocd app sync <app>
argocd app rollback <app> <history-id>
```

Terraform

```
terraform fmt -recursive
terraform init
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
terraform state list
```

Fast answer frameworks

Question type	Use this structure
Troubleshooting	Impact -> recent change -> system path -> evidence -> mitigation -> prevention
Design	Requirements -> architecture -> security -> reliability -> scale -> cost -> operations -> tradeoffs
Tool comparison	What it does -> strengths -> weaknesses -> when I would use it
Security	Threat -> control -> detection -> response -> audit
Incident	Customer impact -> timeline -> mitigation -> root/contributing factors -> action items
Behavioral	STAR: Situation -> Task -> Action -> Result, then lesson learned

High-value tricks to remember answers

Kubernetes objects: Deployment creates ReplicaSet creates Pods; Service selects Pods; Ingress routes to Service.

Probes: Liveness restarts, readiness receives traffic, startup protects slow boot.

Autoscaling: HPA scales pods, VPA recommends/changes resources, Karpenter scales nodes.

GitOps: Git is desired state; controller reconciles actual state.

Helm: Chart is package, templates are blueprints, values are knobs, release is installed instance.

AWS identity: IAM authenticates AWS access; RBAC authorizes Kubernetes access; IRSA gives pods AWS roles.

Observability: Metrics say what, logs say why, traces say where.

SRE: SLO tells target, error budget tells how much risk remains.

Networking: DNS name -> load balancer -> ingress -> service -> endpoints -> pod.

Security: Least privilege, short-lived credentials, encrypted secrets, audited access.

Final interviewer checklist

Before answering any advanced DevOps interview question, mentally check: What is the customer impact? What changed recently? What evidence proves it? What is the safest mitigation? What prevents recurrence? What security or cost tradeoff exists? What metric proves success?